

APEX PLANET INTERNSHIP

TASK 2 - NETWORK SECURITY & SCANNING

Detailed Technical Report

Targets

Metasploitable2 - 192.168.56.101

VulnWeb test target - testaspnet.vulnweb.com / 44.238.29.244

Assessment Workstation: Kali Linux

Assessment period represented in retained logs: 19-21 September 2026

Prepared from retained terminal sessions, screenshots, Nessus results,
Wireshark captures, and the full working conversation.

CONFIDENTIAL TRAINING / INTERNSHIP EVIDENCE

Document Control

Item	Value
Document	Apex Planet Internship - Task 2 Network Security & Scanning
Workstation	Kali Linux in Oracle VirtualBox
Controlled target	Metasploitable2 - 192.168.56.101
Public training target	testaspnet.vulnweb.com - observed as 44.238.29.244
Primary scanner	Nmap 7.99
Vulnerability scanner	Tenable Nessus 10.12.4
Packet analyzer	Wireshark
Traffic generator / SYN analysis	hping3 3.0.0-alpha-2
Firewall	iptables on Metasploitable2
Evidence basis	Retained Kali terminal logs, screenshots, Nessus dashboard results, and Wireshark captures supplied/recorded during the work

Evidence Basis and Accuracy Policy

This report is intentionally evidence-led. It separates commands that were actually executed and outputs that were actually observed from recommended follow-up steps. Where an output was not retained in the copied command log, the report does not manufacture a value. This is particularly important for the exact Low-severity count in Nessus and for the Metasploitable2 UDP scan output, which were not fully transcribed in the retained terminal text.

Evidence source: The retained session log contains 1,234 lines of copied terminal history covering DNS, hping3, Nmap, Nessus, interface configuration, Telnet, WHOIS, Nslookup, and VulnWeb activity. Key examples include the successful DNS response, SYN test, firewall state changes, service/version detection, and Nessus plugin initialization.

Contents

1. Executive Summary
2. Objectives
3. Scope and Target Selection
4. Lab Architecture and Network Setup
5. Chronological Troubleshooting and Recovery
6. Passive Reconnaissance - VulnWeb
7. Active Reconnaissance
8. Port and Service Scanning - Nmap
9. Vulnerability Assessment - Nessus Essentials
10. Wireshark Traffic Analysis
11. TCP SYN Analysis with hping3
12. Firewall Basics and Port-Blocking Demonstration
13. Findings and Security Implications
14. Limitations and Lessons Learned
15. Evidence and Submission Mapping

16. Five-Minute Demonstration Plan
17. Conclusion
18. References
19. Appendix A - Core Commands
20. Appendix B - Selected Raw Outputs

1. Executive Summary

This practical implemented an end-to-end network-security assessment workflow in a two-target design. A deliberately vulnerable public training target (VulnWeb) was used for public passive reconnaissance, while the intentionally vulnerable Metasploitable2 virtual machine was used for active reconnaissance, Nmap enumeration, Nessus vulnerability assessment, Wireshark traffic analysis, controlled TCP SYN analysis, and firewall testing inside an isolated Host-Only network.

The work began with a connectivity problem rather than a clean lab. Kali had a NAT interface for Internet access but its Host-Only interface did not initially have an IPv4 address, and Metasploitable2 similarly lacked a usable IPv4 address. Temporary manual addressing did not immediately produce connectivity. A persistent NetworkManager profile was subsequently created for Kali and the private lab was restored to a stable state at 192.168.56.102/24 (Kali) and 192.168.56.101/24 (Metasploitable2). Successful ping tests and a two-host subnet sweep verified the recovered lab.

The reconnaissance and scanning phase generated eight preserved Nmap result files: four for Metasploitable2 and four for VulnWeb. Metasploitable2 exposed 23 open TCP services within the default 1,000-port set and was fingerprinted as Linux 2.6.x. VulnWeb exposed TCP/80 and was identified as Microsoft IIS 8.5 with ASP.NET headers. OS detection on the public target was explicitly inconclusive.

Nessus Essentials was installed and initialized after troubleshooting an unavailable direct package URL, an incomplete package/service state, and a missing systemd unit. The Nessus daemon was successfully started directly, its plugin set reached 323,592 loaded plugins in the retained session, and the local web interface was confirmed on TCP/8834. A Basic Network Scan against Metasploitable2 produced a dashboard result reporting 59 vulnerabilities. The user-provided dashboard examples included Critical, High, and Medium findings; the exact Low count was not retained in text and is therefore not invented here.

Wireshark was used for HTTP, FTP, DNS and SYN traffic. HTTP was generated with curl, FTP authentication was successfully performed against the lab VM, DNS included a failed test followed by a successful localhost A-record query, and a bounded hping3 test generated 20 SYN probes with 20 SYN-ACK responses. Finally, iptables was used to DROP TCP/80 and ACCEPT TCP/22, with Nmap demonstrating the resulting port-state change from open to filtered for port 80 while SSH remained open.

2. Objectives

1. Demonstrate passive reconnaissance using WHOIS, Nslookup, Google Dorking, and Shodan where public information is applicable.
2. Demonstrate active reconnaissance using reachability checks, ping sweep, and banner grabbing.
3. Perform Nmap TCP SYN (-sS) and UDP (-sU) scans.
4. Perform Nmap service/version detection (-sV) and OS detection (-O).
5. Preserve scan outputs and build an auditable Nmap scan record.
6. Set up Nessus Essentials and assess the controlled Metasploitable2 VM.

7. Capture and analyze HTTP, FTP, and DNS traffic with Wireshark.
8. Demonstrate plaintext FTP authentication exposure in the controlled lab.
9. Analyze TCP SYN behavior using a bounded hping3 test.
10. Configure basic iptables allow/deny rules and verify them with Nmap.
11. Package evidence into a professional GitHub repository, detailed report, and five-minute demonstration.

3. Scope and Target Selection

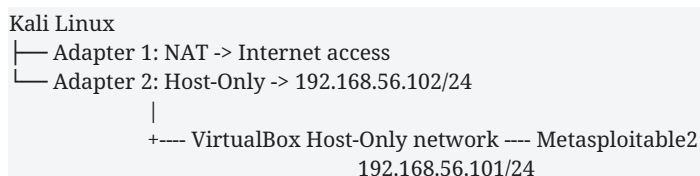
Target	Role	Address	Activities
Metasploitable2	Controlled vulnerable VM / private lab	192.168.56.101	Active recon, Nmap, Nessus, Wireshark, hping3, iptables
VulnWeb test application	Public training target	testaspnet.vulnweb.com / 44.238.29.244	WHOIS, Nslookup, Google Dorking, Shodan, reachability, banner grabbing, Nmap

3.1 Why two targets were necessary

Passive reconnaissance depends on information that is already published or externally indexed. Metasploitable2 was deliberately kept inside a Host-Only network, so applying WHOIS, public search indexing, or Shodan to 192.168.56.101 would not represent the technique correctly. VulnWeb therefore served as the public-facing training target for the passive phase, while Metasploitable2 served as the controlled target for active and defensive laboratory tasks.

Professional explanation used for the demo: “I used two targets because passive and active reconnaissance rely on different sources of information. Metasploitable2 is an isolated private-lab VM, so public passive-recon sources are not representative for that host. I therefore used the authorized public training target for passive reconnaissance and the private VM for controlled scanning and defensive testing.”

4. Lab Architecture and Network Setup



4.1 Initial state

The initial Kali interface inspection showed eth0 on 10.0.2.15/24 and eth1 present but without an IPv4 address. NetworkManager reported eth0 connected and eth1 disconnected. Metasploitable2 similarly had an Ethernet interface without a usable IPv4 address, showing only a link-local IPv6 address at that stage. This explains why early private-lab pings failed.

```

sudo -i
ifconfig
nmcli device status
  
```

4.2 Temporary addressing attempts

```

# Metasploitable2
sudo ifconfig eth0 192.168.56.101 netmask 255.255.255.0 up
  
```

```
# Kali
sudo ip addr replace 192.168.56.102/24 dev eth1
```

The `ip addr replace` operation is a kernel-level address assignment. It can be useful for a quick test, but it does not create a persistent NetworkManager connection profile. Early pings to 192.168.56.101 still returned 100% packet loss, so the configuration was not yet operational.

4.3 Persistent Kali configuration

```
nmcli connection add type ethernet ifname eth1 con-name "Metasploitable-Lab" \
  ipv4.method manual ipv4.addresses 192.168.56.102/24 ipv6.method disabled

nmcli connection up "Metasploitable-Lab"
```

This created a saved NetworkManager profile. Subsequent verification showed eth1 connected as `Metasploitable-Lab` and holding 192.168.56.102/24.

4.4 Connectivity proof

```
ping -c 4 192.168.56.101
```

Observed result: 4 packets transmitted, 4 received, 0% packet loss; average RTT approximately 2.823 ms in the retained session. The successful ping was followed by an Nmap subnet sweep that found exactly two active lab hosts: 192.168.56.101 and 192.168.56.102.

5. Chronological Troubleshooting and Recovery

1. VirtualBox networking was confirmed with NAT on the primary adapter and Host-Only networking on the secondary adapter.
2. Kali eth1 was initially present but had no IPv4 address; Metasploitable2 eth0 likewise lacked IPv4.
3. Temporary addresses were assigned to both VMs.
4. Initial connectivity tests returned 100% packet loss, demonstrating that having an interface name alone was not sufficient.
5. Kali was reconfigured with a persistent NetworkManager profile named `Metasploitable-Lab`.
6. Metasploitable2 was restored to 192.168.56.101/24 and the Host-Only path became reachable.
7. Ping and Nmap ping-sweep evidence established a working two-host private lab.
8. The project files were initially written directly into `/root`; the `/root/apex-planet-lab` directory was later confirmed empty and then populated with the Task-2 structure.
9. Eight Nmap text outputs were moved into `Task-2-Network-Security/nmap/` rather than rerunning the scans.
10. GitHub browser uploads later showed that the authenticated account lacked push access to the selected repository/branch, so the web UI instructed the user to fork the repository. This was a repository-access issue, not a problem with the evidence files.

5.1 Practical troubleshooting observations retained for the final record

Issue	What was observed	How it was handled
Private lab no IPv4	eth1 / eth0 initially lacked usable IPv4	Assigned temporary IPs, then created persistent NetworkManager profile on Kali
Initial ping failure	100% packet loss to 192.168.56.101	Verified interface states and corrected Host-Only addressing
WHOIS against private VM	No meaningful public registration record	Skipped as inapplicable; used VulnWeb

		for public passive recon
Raw Telnet bytes in netcat	Binary-looking negotiation characters	Confirmed with Telnet client and documented as protocol negotiation, not a backdoor
DNS SERVFAIL	metasploitable.localdomain returned SERVFAIL	Ran a second legitimate query for localhost A, which returned NOERROR
UDP stream looked like gibberish	Raw DNS bytes shown by Follow UDP Stream	Used Wireshark DNS decoder/filter instead of treating raw bytes as text
Nessus service unit missing	systemctl reported no `nessusd.service`	Used `/opt/nessus/sbin/nessus-service -D` and verified port 8834
Nessus export restricted	UI requested an upgrade for export	Retained screenshots of the completed scan/results; did not fabricate a report file
zsh history warning	`zsh: corrupt history file /root/.zsh_history`	Treated as unrelated shell-history noise; it did not affect the security tests

6. Passive Reconnaissance - VulnWeb

6.1 WHOIS - domain

```
whois vulnweb.com
```

The retained WHOIS record identified Gandi SAS as registrar, creation date 2010-06-14, registry expiry 2027-06-14, four AWS DNS name servers, and DNSSEC marked unsigned. These are passive registration/infrastructure observations, not vulnerability findings.

6.2 WHOIS - target IP

```
whois 44.238.29.244
```

ARIN returned an Amazon.com / Amazon Web Services allocation, including the 44.192.0.0/10 network shown in the record. This was used only as passive ownership/infrastructure context.

6.3 Nslookup

```
nslookup testaspnet.vulnweb.com
```

Observed: `testaspnet.vulnweb.com` resolved to 44.238.29.244 using the local resolver shown in the session.

6.4 Google Dorking

```
site:vulnweb.com
```

This browser-based search was used to identify publicly indexed resources associated with the training domain. It was not applied to the private Metasploitable2 address.

6.5 Shodan

The resolved public IP, 44.238.29.244, was queried in Shodan. The observed IP search returned host/infrastructure information, which complements the DNS and WHOIS results. The report does not claim that every domain-level Shodan query returned the same result.

7. Active Reconnaissance

7.1 Host reachability - VulnWeb

```
ping -c 4 testaspnet.vulnweb.com
```

Observed: 4 transmitted, 4 received, 0% packet loss. The second IP-based ping sent 12 packets before manual interruption and likewise recorded 0% packet loss.

7.2 Host discovery - VulnWeb

```
sudo nmap -sn testaspnet.vulnweb.com
```

Nmap reported the host as up and recorded the reverse-DNS name `ec2-44-238-29-244.us-west-2.compute.amazonaws.com`.

7.3 Ping sweep - private lab

```
sudo nmap -sn 192.168.56.0/24
```

Exactly two hosts were identified as up: 192.168.56.101 and 192.168.56.102. The Metasploitable2 MAC address was shown as an Oracle VirtualBox virtual NIC.

7.4 Banner grabbing - Metasploitable2 Telnet

```
nc -v 192.168.56.101 23
```

Netcat confirmed TCP/23 open. The raw bytes displayed after connection were Telnet negotiation data. A follow-up Telnet client connection returned the readable training banner and Linux system information. The supplied `msfadmin` account was accepted and produced a remote shell. This is correctly described as authorized service authentication with known lab credentials, not as a Telnet vulnerability exploit or authentication bypass.

```
telnet 192.168.56.101 23
```

7.5 Banner grabbing - VulnWeb HTTP

```
nc -v 44.238.29.244 80
HEAD / HTTP/1.0
```

Observed headers included `HTTP/1.1 200 OK`, `Server: Microsoft-IIS/8.5`, and `X-Powered-By: ASP.NET`. Additional headers showed a content length of 701 bytes and a Last-Modified timestamp.

8. Port and Service Scanning - Nmap

Method note: Nmap describes `open` as an application accepting traffic, `closed` as a reachable port with no listener, and `filtered` as a state where filtering prevents Nmap from determining openness. `open | filtered` must not be reported as confirmed open.

8.1 Metasploitable2 - TCP SYN scan

```
sudo nmap -sS 192.168.56.101 -oN metasploitable-tcp.txt
```

Result: 977 closed TCP ports and 23 open TCP services in the default 1,000-port scan.

Port	Service	State	Observed version / identification
21	FTP	open	vsftpd 2.3.4
22	SSH	open	OpenSSH 4.7p1
23	Telnet	open	Linux telnetd

25	SMTP	open	Postfix smtpd
53	DNS	open	ISC BIND 9.4.2
80	HTTP	open	Apache httpd 2.2.8
111	RPCbind	open	RPC #100000
139	NetBIOS	open	Samba smbd 3.X-4.X
445	SMB	open	Samba smbd 3.X-4.X
512	exec	open	netkit-rsh rexecd
513	login	open	rlogind/login service
514	shell	open	Netkit rshd
1099	Java RMI	open	GNU Classpath grmiregistry
1524	bind shell	open	Metasploitable root shell
2049	NFS	open	NFS 2-4
2121	FTP	open	ProFTPD 1.3.1
3306	MySQL	open	MySQL 5.0.51a
5432	PostgreSQL	open	PostgreSQL 8.3.0-8.3.7
5900	VNC	open	VNC protocol 3.3
6000	X11	open	access denied
6667	IRC	open	UnrealIRCd
8009	AJP	open	Apache Jserv 1.3
8180	HTTP	open	Apache Tomcat/Coyote JSP engine 1.1

8.2 Metasploitable2 - service/version detection

```
sudo nmap -sV 192.168.56.101 -oN metasploitable-services.txt
```

The service/version results listed above were preserved in `metasploitable-services.txt`. The retained output additionally identified hostnames `metasploitable.localdomain` and `irc.Metasploitable.LAN` and characterized the system as Unix/Linux.

8.3 Metasploitable2 - OS detection

```
sudo nmap -O 192.168.56.101 -oN metasploitable-os.txt
```

Observed OS fingerprint: general purpose; Running Linux 2.6.X; OS details Linux 2.6.9-2.6.33; network distance 1 hop. The result is retained as Nmap fingerprint output, not as a kernel patch-level guarantee.

8.4 Metasploitable2 - combined verification

```
sudo nmap -sS -sV -O 192.168.56.101
```

The combined scan reproduced the service inventory and Linux OS fingerprint in one run, providing a compact verification view for the five-minute demonstration.

8.5 Metasploitable2 - UDP scan

```
sudo nmap -sU 192.168.56.101 -oN metasploitable-udp.txt
```

The raw output file was created and preserved. The copied session text did not retain the completed result block, so this report does not invent the UDP service list or state summary.

8.6 VulnWeb - required Nmap scans

```
sudo nmap -sS 44.238.29.244 -oN vulnweb-tcp.txt
sudo nmap -sU 44.238.29.244 -oN vulnweb-udp.txt
sudo nmap -sV 44.238.29.244 -oN vulnweb-services.txt
sudo nmap -O 44.238.29.244 -oN vulnweb-os.txt
```

Test	Observed result
TCP SYN (-sS)	80/tcp open http; 999 other scanned TCP ports shown as filtered/no-response
UDP (-sU)	1,000 scanned UDP ports reported open filtered because there was no response
Version (-sV)	80/tcp open http - Microsoft IIS httpd 8.5; Service Info OS: Windows
OS (-O)	No exact OS match. Nmap explicitly warned results may be unreliable because it could not find at least one open and one closed port; guesses included AT&T embedded/VirtualBox/Slirp/QEMU. These guesses are not treated as confirmed OS facts.

Command correction captured during troubleshooting: The command `nmap 44.238.29.244 80` was initially interpreted by Nmap as two targets: the real IP and the numeric target `80` (0.0.0.80). The correct way to restrict the scan to TCP/80 is `nmap -p 80 44.238.29.244`.

9. Vulnerability Assessment - Nessus Essentials

9.1 Installation detection and package troubleshooting

```
dpkg -l | grep -i nessus
```

Initially there was no Nessus package listed. A direct Tenable package URL was then attempted and returned HTTP 404. The package was subsequently downloaded through the web interface as `Nessus-10.12.4-ubuntu1604\_amd64.deb` and found at `/home/kali/Downloads/` with a retained size of 73 MB.

```
dpkg -i /home/kali/Downloads/Nessus-10.12.4-ubuntu1604_amd64.deb
```

After installation, `dpkg -l | grep -i nessus` showed package version 10.12.4 in an `iF` state in the retained session, and `systemctl status nessusd` reported that the unit was not found. The Nessus server binaries were, however, present under `/opt/nessus/sbin/`.

9.2 Direct Nessus daemon startup

```
/opt/nessus/sbin/nessus-service -D
```

The daemon reported that Nessus had started. Plugin processing subsequently reached 100% and the retained terminal session recorded 323,592 plugins loaded in 57 seconds.

```
ss -lntp | grep 8834
```

Observed: Nessus listening on 0.0.0.0:8834 and [::]:8834. The local interface was then accessed at `https://127.0.0.1:8834`.

9.3 Metasploitable2 vulnerability scan

A Basic Network Scan was created for 192.168.56.101 with the scan name `Metasploitable2-Apex-Task2`. The completed dashboard reported 59 vulnerabilities.

Severity	Representative finding reported by the user	Observed CVSS
Critical	VNC Server Password Vulnerability	10.0
Critical	Canonical Ubuntu Linux issues	10.0
Critical	SSL Version 2/3 and Bind Shell Backdoor	9.8
High	NFS Shares World Readable	7.5
High	rlogin Service Detection	7.5
Medium	TLS Version 1.0 Protocol	6.5
Low	Exact finding/count not retained in copied terminal text	Not reported here

Severity handling: Tenable Nessus exposes severity as part of scan results. The report uses the CVSS values explicitly visible in the user-provided scan description and does not infer a Low result that was not transcribed.

9.4 Nessus export limitation

When the user attempted to export the scan, the UI presented an upgrade requirement. Therefore, screenshots of the scan summary and findings are the primary Nessus evidence for this submission. The report does not claim an exported PDF/CSV that was not successfully obtained.

10. Wireshark Traffic Analysis

10.1 Capture setup

```
wireshark
```

The retained Wireshark process log shows capture start at approximately 02:01:37 and capture stop at approximately 02:51:59, with the file path ``/tmp/wireshark_eth10MQ5V3.pcapng``. GUI warnings about desktop portal registration were present but did not prevent capture; the log explicitly recorded Capture Start and Capture Stop.

10.2 HTTP traffic

```
curl http://192.168.56.101
```

The response contained the Metasploitable2 web landing page and links to TWiki, phpMyAdmin, Mutillidae, DVWA, and WebDAV. This provided a simple application-layer HTTP transaction for Wireshark analysis.

10.3 FTP traffic and plaintext authentication

```
ftp 192.168.56.101
```

The FTP service answered ``220 (vsFTPD 2.3.4)``. The lab account ``msfadmin`` authenticated successfully and the session entered the FTP prompt. For packet analysis, the FTP authentication commands were filtered in Wireshark. The captured credential material should be treated as lab-only evidence and must not be uploaded to a public repository in readable form.

```
ftp.request.command == "USER" || ftp.request.command == "PASS"
```

10.4 DNS traffic

```
dig @192.168.56.101 metasploitable.localdomain
dig @192.168.56.101 localhost A
```

The first query returned SERVFAIL. The second query returned NOERROR with one A-record answer: localhost -> 127.0.0.1. This was the successful DNS packet-analysis case used for the report.

```
dns
```

The `dns` Wireshark display filter is preferable to “Follow UDP Stream” for explaining DNS because raw UDP-stream output may appear as non-printable or encoded bytes. Wireshark’s DNS protocol decoder exposes the query name, record type, and answer fields directly.

11. TCP SYN Analysis with hping3

11.1 Tool verification

```
hping3 --version
```

Observed: hping3 version 3.0.0-alpha-2.

11.2 Bounded SYN test

```
sudo hping3 -S -p 80 -c 20 -i u100000 192.168.56.101
```

The test generated 20 SYN probes against TCP/80 of the private Metasploitable2 VM. Every probe received a response with TCP flags `SA` (SYN+ACK). The first run reported 20 transmitted, 20 received, 0% packet loss, with RTT 1.5/4.0/21.6 ms; the second run reported 1.0/2.0/2.9 ms.

11.3 Interpretation

The observed SA responses demonstrate the target’s response to SYN requests and confirm that TCP/80 was accepting the initial connection request at the time of the test. This was a bounded traffic-analysis exercise, not evidence that the machine was overwhelmed or taken offline by a denial-of-service condition.

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

12. Firewall Basics and Port-Blocking Demonstration

12.1 Baseline

```
sudo nmap -p 80 192.168.56.101
```

Baseline observation: 80/tcp open http.

12.2 Deny rule

```
iptables -I INPUT -p tcp --dport 80 -j DROP
```

After the rule was applied on Metasploitable2, the verification scan returned 80/tcp filtered http. The host remained reachable, but the specific TCP/80 traffic was silently dropped by the INPUT chain rule.

12.3 Allow rule

```
iptables -I INPUT -p tcp --dport 22 -j ACCEPT  
sudo nmap -p 22,80 192.168.56.101
```

Verification showed 22/tcp open ssh while 80/tcp remained filtered. This single scan demonstrated both allow and deny behavior.

12.4 Rule cleanup

```
iptables -D INPUT -p tcp --dport 22 -j ACCEPT
iptables -D INPUT -p tcp --dport 80 -j DROP
iptables -L INPUT -n --line-numbers
```

These commands were provided as the post-test cleanup sequence so the temporary demonstration rules could be removed. The retained copied log does not explicitly show the cleanup output, so the report treats cleanup as a documented recovery step rather than claiming observed execution.

13. Findings and Security Implications

Area	Observation from evidence	Security implication	Recommended control
Service exposure	Metasploitable2 exposed 23 TCP services	Large attack surface and multiple legacy protocols	Disable unnecessary services and restrict access by network zone
Telnet	TCP/23 exposed and remote login accepted with known lab credentials	Plaintext legacy remote administration and weak/default credentials increase exposure	Disable Telnet; use SSH with strong authentication
FTP	vsftpd 2.3.4 exposed; lab login succeeded	FTP authentication can be observed in plaintext traffic	Replace FTP with SFTP/FTPS and eliminate default credentials
NFS	NFS exposed; Nessus reported world-readable NFS shares	File exports may be readable beyond intended users	Restrict exports and filesystem permissions
rlogin/r-services	512-514 exposed; Nessus reported rlogin service detection	Legacy trust-based remote access is unsafe on modern networks	Disable r-services; use SSH
VNC	Nessus reported VNC Server Password Vulnerability, CVSS 10.0	Remote desktop authentication exposure	Disable VNC where unnecessary and require strong protected access
Legacy TLS	Nessus reported TLS 1.0 Protocol, CVSS 6.5	Legacy protocol can weaken transport security	Disable TLS 1.0 and use modern TLS versions
Public target fingerprinting	VulnWeb identified IIS 8.5 / ASP.NET; OS scan inconclusive	Technology fingerprinting provides useful attack-surface context, but OS inference must be bounded by evidence	Minimize unnecessary server headers and maintain supported platform versions
Firewall control	Port 80 changed from open to filtered after DROP; SSH remained open after ACCEPT	Host firewall can materially alter attack visibility and reachability	Use explicit allow/deny rules and verify from a separate host

14. Limitations and Lessons Learned

- Passive reconnaissance is target-dependent. A private Host-Only VM is not a substitute for a public Internet target when demonstrating WHOIS, search-indexing, or Shodan.
- OS detection is fingerprinting, not ground truth. On VulnWeb, Nmap explicitly reported the result as unreliable and no exact OS match was found.
- UDP scanning is inherently less definitive than TCP SYN scanning in many cases. “open | filtered” means Nmap could not distinguish the two states from the available responses.
- Telnet banner output through Netcat initially appeared as binary garbage because of protocol negotiation; using a Telnet-aware client made the banner readable.
- DNS raw-stream output can look unreadable because DNS is a binary protocol. Packet decoding is more meaningful than reading raw byte streams.

- The first Metasploitable2 DNS query returned SERVFAIL. The successful `localhost A` query provided a clean positive DNS example without changing the lab configuration.
- The Nessus daemon was functional even though a `nessusd.service` unit was not present. Direct startup using the installed Nessus service binary was sufficient to bring the scanner online.
- The exact Low-severity Nessus count was not transcribed into the retained text. It should be captured from the dashboard before final submission if the mentor requires explicit Low examples.
- The retained Wireshark capture is sensitive because it contains lab authentication traffic. Credential-bearing PCAP/PNG evidence should not be published to a public repository without redaction or explicit approval.
- The `zsh` corrupt-history warning is unrelated to the network/security tests and should not be presented as a lab failure.

15. Evidence and Submission Mapping

Requirement	Evidence to submit	Current state
Passive recon	WHOIS / Nslookup / Google Dorking / Shodan screenshots or captured notes	Completed and documented
Ping sweep	Nmap -sn screenshot	Completed
Banner grabbing	Telnet and HTTP banner screenshots	Completed
Nmap TCP/UDP	8 raw .txt files + selected screenshots	Completed; files preserved
Service/version + OS	Nmap -sV / -O outputs	Completed
Nessus vulnerability assessment	Nessus dashboard screenshots + detailed report section	Scan completed; exact Low entry still needs dashboard capture if required
HTTP/FTP/DNS Wireshark	PNG screenshots + PCAP/PCAPNG evidence	Captured during practical
FTP plaintext credentials	Wireshark filtered USER/PASS screenshot	Captured; keep credentials out of public GitHub
SYN analysis	hping3 output + Wireshark SYN screenshot	Completed
iptables allow/deny	Before/after Nmap screenshots + rule evidence	Completed
Detailed report	Task-2-Network-Security-Report.pdf	Prepared
GitHub repository	README.md + nmap + wireshark + report + screenshots folders	Structure prepared
Five-minute demo	Final screen-recording/video link	Script/flow prepared

16. Five-Minute Demonstration Plan

Time	Content	What to say
0:00-0:30	Architecture + why two targets	Metasploitable2 is the isolated lab; VulnWeb is the public target for passive recon.
0:30-1:00	WHOIS / Nslookup / Google / Shodan	Show public intelligence only; explain why private-lab passive recon is limited.
1:00-1:45	Ping sweep + banner grabbing	Show two lab hosts and Telnet/HTTP service identification.
1:45-2:30	Nmap + findings	Show -sS, -sV, -O, -sU and highlight the 23-service Metasploitable2 inventory.
2:30-3:30	Nessus + Wireshark	Show 59-finding dashboard and

		representative Critical/High/Medium examples, then HTTP/FTP/DNS evidence.
3:30-4:15	hping3 + SYN analysis	Show bounded 20-SYN test and SYN/SYN-ACK interpretation.
4:15-4:45	iptables before/after	Show port 80 open -> filtered and port 22 open after ACCEPT.
4:45-5:00	Conclusion	Summarize the complete workflow and evidence package.

17. Conclusion

The practical evolved from a broken private-lab connection into a working, evidence-backed network-security assessment environment. The important technical outcome is not only the final scan results but the repeatable troubleshooting path: identify interface state, assign addresses correctly, verify reachability, inventory services, assess vulnerabilities, inspect actual traffic, test TCP behavior, and demonstrate how host firewall rules change observed exposure.

The final methodology is deliberately split by target role. VulnWeb supplies the public information needed for passive reconnaissance, while Metasploitable2 supplies the controlled attack surface needed for Nmap, Nessus, Wireshark, hping3, and iptables work. This separation keeps the report technically coherent and prevents public passive-recon techniques from being misapplied to an isolated private IP.

The retained evidence is sufficient to support the major practical requirements. The only material data gap identified in the copied command log is the exact Low-severity Nessus finding/count; that item should be taken directly from the Nessus dashboard before final submission if the instructor requires all four severity categories to be explicitly exemplified.

18. References

- Nmap Network Scanning - official Nmap guide: <https://nmap.org/book/>
- Nmap Port Scanning Basics: <https://nmap.org/book/man-port-scanning-basics.html>
- Nmap Port Scanning Techniques: <https://nmap.org/book/man-port-scanning-techniques.html>
- Nmap Reference Guide: <https://nmap.org/book/man.html>
- Tenable Nessus 10.12 Getting Started: <https://docs.tenable.com/nessus/Content/GettingStarted.htm>
- Tenable Nessus Severity: https://docs.tenable.com/nessus/10_12/Content/Severity.htm
- Tenable Nessus Scan Summary: <https://docs.tenable.com/nessus/Content/ViewScanSummary.htm>
- Tenable Nessus 10.12 User Guide: https://docs.tenable.com/nessus/10_12/Content/PDF/Nessus_10_12.pdf
- Packt - Active Reconnaissance reference used during the practical:
<https://subscription.packtpub.com/book/security/9781837630639/9/ch09lv11sec51/active-reconnaissance>

Appendix A - Core Commands

Lab connectivity

```
sudo -i
ifconfig
nmcli device status
nmcli connection add type ethernet ifname eth1 con-name "Metasploitable-Lab" ipv4.method manual ipv4.addresses
192.168.56.102/24 ipv6.method disabled
nmcli connection up "Metasploitable-Lab"
ping -c 4 192.168.56.101
```

Private-lab discovery

```
sudo nmap -sn 192.168.56.0/24
nc -v 192.168.56.101 23
telnet 192.168.56.101 23
```

VulnWeb passive / active

```
whois vulnweb.com
whois 44.238.29.244
nslookup testaspnet.vulnweb.com
ping -c 4 testaspnet.vulnweb.com
sudo nmap -sn testaspnet.vulnweb.com
nc -v 44.238.29.244 80
```

Metasploitable2 Nmap

```
sudo nmap -sS 192.168.56.101 -oN metasploitable-tcp.txt
sudo nmap -sU 192.168.56.101 -oN metasploitable-udp.txt
sudo nmap -sV 192.168.56.101 -oN metasploitable-services.txt
sudo nmap -O 192.168.56.101 -oN metasploitable-os.txt
sudo nmap -sS -sV -O 192.168.56.101
```

VulnWeb Nmap

```
sudo nmap -sS 44.238.29.244 -oN vulnweb-tcp.txt
sudo nmap -sU 44.238.29.244 -oN vulnweb-udp.txt
sudo nmap -sV 44.238.29.244 -oN vulnweb-services.txt
sudo nmap -O 44.238.29.244 -oN vulnweb-os.txt
```

Nessus

```
dpkg -l | grep -i nessus
dpkg -i /home/kali/Downloads/Nessus-10.12.4-ubuntu1604_amd64.deb
/opt/nessus/sbin/nessus-service -D
ss -lntp | grep 8834
```

Traffic generation / Wireshark

```
curl http://192.168.56.101
ftp 192.168.56.101
dig @192.168.56.101 metasploitable.localdomain
dig @192.168.56.101 localhost A
ftp.request.command == "USER" || ftp.request.command == "PASS"
dns
```

SYN analysis

```
hping3 --version
sudo hping3 -S -p 80 -c 20 -i u100000 192.168.56.101
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

Firewall demonstration

```
sudo nmap -p 80 192.168.56.101
iptables -I INPUT -p tcp --dport 80 -j DROP
sudo nmap -p 80 192.168.56.101
iptables -I INPUT -p tcp --dport 22 -j ACCEPT
sudo nmap -p 22,80 192.168.56.101
iptables -D INPUT -p tcp --dport 22 -j ACCEPT
iptables -D INPUT -p tcp --dport 80 -j DROP
```

Appendix B - Selected Raw Outputs

B.1 Successful private-lab connectivity

```
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=4.42 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=2.13 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=2.58 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=2.16 ms

4 packets transmitted, 4 received, 0% packet loss
```

B.2 Successful DNS query

```
status: NOERROR
QUESTION: localhost. IN A
ANSWER: localhost. 604800 IN A 127.0.0.1
SERVER: 192.168.56.101#53(192.168.56.101) (UDP)
```

B.3 SYN analysis

```
HPING 192.168.56.101 (eth1 192.168.56.101): S set, 40 headers + 0 data bytes
... flags=SA ...
20 packets transmitted, 20 packets received, 0% packet loss
```

B.4 Firewall before / after

```
BEFORE
80/tcp open http

AFTER DROP
80/tcp filtered http

ALLOW 22 + DENY 80
22/tcp open ssh
80/tcp filtered http
```

B.5 Nessus initialization

```
nessus-service ... Started
Processing the Nessus plugins...
[=====] 100%
323592 plugins loaded (57sec)
```

B.6 Nmap files preserved

```
metasploitable-tcp.txt; metasploitable-udp.txt; metasploitable-services.txt; metasploitable-os.txt; vulnweb-tcp.txt; vulnweb-udp.txt; vulnweb-services.txt; vulnweb-os.txt
```

Publishing note: Do not place a readable FTP credential screenshot or credential-bearing PCAP/PCAPNG in a public GitHub repository. Keep those artifacts private or redact them before publication.